

# 浙江树人学院信息科技学院 人工智能 期末大作业

(2025/2026学年第二学期)

| 项目    | 内容                   |
|-------|----------------------|
| 大作业题目 | Gradient Lab · 渐变实验室 |
| 专业    | 物联网工程                |
| 班级    | 物联网 231              |
| 姓名    | 陈宇杰                  |
| 学号    | 202305311115         |

## 一、项目概述与工具选型

### 网页项目主题

**Gradient Lab (渐变实验室)** — 一个面向设计师和前端开发者的渐变色实时预览与生成工具。

### 项目核心功能

- 渐变实时预览** — 支持线性/径向两种渐变模式，角度可拖拽调节
- 多颜色点编辑** — 自由添加、删除、调色，实时反映到预览区
- 灵感预设库** — 内置 8 组精选配色（日落、海洋、极光、霓虹、紫夜、森林、糖果、火红），一键切换
- 智能随机生成** — 基于 HSL 色彩空间的 5 种配色算法（类似色、互补色、三角色、暖色系、冷色系），每次点击生成全新渐变
- 一键复制 CSS** — 自动生成标准 CSS gradient 代码，一键复制到剪贴板

选用 AI 工具及选择理由

| 工具名称                                      | 选择原因（优势）                                                                          |
|-------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Hermes Agent<br/>(DeepSeek v4 Pro)</b> | 主力协作工具。支持微信端直接对话，代码生成质量高，能理解复杂需求并一次性生成完整可用的 HTML 文件。全程通过自然语言提示词驱动开发。              |
| <b>Chrome DevTools</b>                    | 调试工具。通过 Playwright MCP 集成，AI 可直接在浏览器中截图验证效果、检查控制台错误、模拟用户交互，实现"AI 写代码→浏览器自动测试"的闭环。 |

二、第一轮原始提词

创作思路

在浏览课程大作业要求后，发现此前完成的 FrigoBrain (Android APP) 和 CCTT (Flutter APP) 均为原生移动应用，不符合"单个 HTML 文件、浏览器直接打开运行"的要求，决定重新构思选题。希望通过一个视觉效果好、功能实用、交互丰富的网页工具来展示 Vibe Coding 的能力。经过与 AI 多轮讨论，在四个候选方案中选择了**渐变实验室**，因为渐变本身就是视觉焦点，天然美观，且功能之间有清晰的逻辑闭环。

第一轮提示词

"不用结合我的背景，不用你给出的技能。想几个好实现的，界面美观惊艳的选题"

AI 给出四个方案：像素画板、声音可视化留声机、3D 翻牌记忆游戏、渐变实验室。

第一轮成品效果评价

四个方案各有特点：像素画板代码量最小但交互偏简单；声音可视化演示效果炸裂但 Web Audio API 调试稍复杂；翻牌游戏经典但创新性不足；渐变实验室最简单

稳定且界面天然好看，四个功能轻松满足评分要求。选择渐变实验室作为最终方案。

---

### 三、多轮 AI 交互迭代日志

---

#### 第 1 轮：需求确认与选题

**1. 优化需求** 用户提供大作业要求文档，询问已有项目（FrigoBrain、CCTT）能否直接用于作业。

**2. 优化提示词**

"你看一下我们目前为止做过的项目，比如frigobrain和cctt，能不能直接用于这次的作业"

**3. AI 返回分析** AI 分析了两个项目的技术栈和产物类型，指出： - FrigoBrain 为 Android Kotlin 原生应用（APK），不符合 HTML 要求 - CCTT 为 Flutter 应用 + Node.js 后端，同样无法在浏览器直接运行 - 两个项目均未使用 Vibe Coding 流程开发，缺少 AI 协作对话记录 - 结论：不能直接提交，但创意可复用

**4. 本轮优化后页面评价** AI 的分析准确、有理有据，一次性厘清了可行性问题，避免了用户走弯路。

---

#### 第 2 轮：选题构思

**1. 优化需求** 确定重新构思选题。用户表示不需要使用 Trae，与 Hermes Agent 的对话即可作为 Vibe Coding 过程。

**2. 优化提示词**

"不用trae，我现在跟你对话就算一种vibe coding。那再构思一个选题吧"

**3. AI 返回** AI 给出四个选题方案： 1. **像素画板** — Canvas 点格子填色 + 导出 PNG 2. **声音可视化留声机** — Web Audio API 频谱可视化 3. **3D 翻牌记忆游戏**

— CSS 3D transform 翻转配对 4. **渐变实验室** — 渐变预览 + 随机生成 + 预设库 + CSS 复制

**4. 本轮优化后页面评价** 四个方案覆盖了不同技术点和交互类型，分析详实，对比维度清晰（实现难度、视觉效果、功能数量）。用户最终选择方案 4。

---

### 第 3 轮：代码生成

**1. 优化需求** 用户选择渐变实验室方案后，需要一次性生成完整可运行的 HTML 文件。

**2. 优化提示词**

"4感觉可行"

**3. AI 返回代码** AI 在确认选题后，加载了 `frontend-design` 技能以获取前端设计最佳实践，然后一次性生成了完整的 HTML 文件（约 450 行），包含：- 暗色主题 + 玻璃质感 UI 设计 - 完整的 CSS 变量系统和响应式布局 - 渐变色实时预览区（支持鼠标拖拽调角度） - 颜色点多选编辑系统（添加/删除/取色器） - 线性/径向模式切换 - 8 组预设渐变 - 基于 HSL 色彩空间的 5 种随机配色算法 - CSS 代码复制 + Toast 提示 - 过渡动画和微交互效果

**4. 本轮优化后页面评价** 代码首次运行即无 JS 报错，所有功能正常工作。页面视觉效果好，暗色主题搭配霓虹渐变色有高级感。预设和随机功能均能正常运作。角度拖拽交互流畅。

---

### 第 4 轮：浏览器验证与调试

**1. 优化需求** 生成代码后需要通过浏览器验证功能是否正常，检查是否存在运行时错误。

**2. 优化提示词**

(AI 自主启动 HTTP 服务器并通过 Playwright 浏览器自动化工具测试页面)

**3. AI 返回** AI 通过浏览器自动化执行了以下测试： - 页面渲染验证（所有 UI 组件正常显示） - 控制台错误检查（仅 `favicon.ico 404`，非功能性错误） - 预设点击切换测试 ☒ 随机生成功能测试 ☒ 线性/径向模式切换测试 ☒ 角度控件显示/隐藏测试 ☒ 页面截图保存为验证依据

全部功能通过验证，无需修改代码。

**4. 本轮优化后页面评价** 首轮代码即达到可提交水平，迭代效率高。浏览器自动化测试替代了人工测试，节省了大量反复调整的时间。

## 四、调试与测试

### 测试案例 1：控制台报错排查

| 步骤    | 内容                                                                                                        |
|-------|-----------------------------------------------------------------------------------------------------------|
| 错误现象  | 浏览器控制台显示 <code>Failed to load resource: the server responded with a status of 404 (File not found)</code> |
| 定位方式  | 通过 Playwright MCP 的 <code>browser_console_messages</code> 工具获取完整错误信息                                      |
| 提示词   | (AI 自主调用浏览器控制台检查)                                                                                         |
| AI 方案 | 识别为浏览器自动请求 <code>favicon.ico</code> 导致的 404，非功能性错误，不影响页面正常运行，无需修复                                         |
| 最终修复  | 确认无害，忽略（单 HTML 文件无需 <code>favicon</code> ）                                                                |

### 测试案例 2：交互功能完整性验证

| 步骤   | 内容                   |
|------|----------------------|
| 错误现象 | 需要验证所有 5 个核心交互功能是否正常 |

| 步骤    | 内容                                                                       |
|-------|--------------------------------------------------------------------------|
| 定位方式  | 通过 Playwright MCP 的 <code>browser_evaluate</code> 工具执行 JavaScript 模拟用户操作 |
| 提示词   | (AI 自主编写测试脚本: 点击预设→检查颜色变化→随机生成→切换径向模式→验证角度控件隐藏)                          |
| AI 方案 | 顺序执行功能测试, 每个步骤返回状态值, 确认预设切换、随机生成、模式切换、角度控件联动均正常                          |
| 最终修复  | 全部通过, 无需修复                                                               |

## 五、成品效果展示

### 网页完整预览

(截图: 渐变实验室主界面, 展示默认紫粉渐变预览区、颜色点编辑栏、类型切换控件、预设库、操作按钮)

### 核心交互功能演示

- 预设切换** — 点击"日落"预设, 预览区瞬间切换为橙红渐变, 颜色点同步更新
- 随机生成** — 点击"随机生成"按钮, 每次生成全新的配色方案, 基于 HSL 算法确保配色协调
- 类型切换** — 点击"径向"按钮, 渐变从线性切换为圆形径向, 角度控件自动隐藏
- CSS 复制** — 点击"复制 CSS", 底部弹出 Toast 提示"☑CSS 已复制到剪贴板"
- 角度拖拽** — 在预览区按住鼠标拖动, 渐变角度实时跟随鼠标方向变化

## 六、Vibe Coding 学习总结与思考

---

### 1. 使用 AI 协作开发和自己手写代码的区别

最大的区别在于**思考方式的重心转移**。手写代码时，大量精力消耗在语法记忆、API 查阅、CSS 调参等机械性工作上；而 Vibe Coding 让你把精力集中在**需求描述、方案决策和质量把关**三个高层级任务上。比如本次开发中，我没有手动写任何一行 CSS 动画代码，而是通过"暗色主题 + 玻璃质感 + 流畅过渡"这样的自然语言描述，让 AI 去实现具体细节。这相当于把 AI 当作一个"无限耐心的初级工程师"，我扮演的是技术主管的角色——定方向、审代码、提修改意见。

但这也带来一个关键挑战：**你必须能判断 AI 输出的质量**。如果自己不具备 HTML/CSS/JS 的基础知识，就无法发现 AI 代码中的问题，更无法提出精准的修改指令。Vibe Coding 不是"不用会编程"，而是"把你从编程细节中解放出来，但要求你具备更高层次的代码鉴赏力和架构判断力"。

### 2. 好的提示词对最终成品的影响

本次开发中，提示词的演进过程让我深刻体会到了提示词质量对输出的决定性影响：

- **模糊提示词** ("给我做个好看的网页") → AI 会返回泛泛的方案，需要多轮澄清
- **带约束的提示词** ("好实现的、界面美观惊艳的、至少 3 个交互功能") → AI 的提案质量和针对性显著提升
- **精准的确认信号** ("4感觉可行") → 虽然简短，但给了 AI 明确的执行方向，AI 立即加载相关技能并全量生成代码

好的提示词有两个特征：**边界清晰**（告诉 AI 不要什么比告诉它要什么更重要）和**验收标准明确**（"好实现"、"界面美观"、"至少 3 个功能"都是可验证的硬性约束）。模糊的需求只会得到模糊的结果。

### 3. 本次作业收获与后续改进思路

**收获：** - 掌握了"对话即开发"的工作流：需求澄清 → 方案讨论 → 代码生成 → 浏览器自动验证 - 认识到 AI 作为"推理引擎"和"执行引擎"的双重价值——它既能帮你分

析方案可行性，也能直接产出代码 - 学会了利用浏览器的自动化测试能力形成"AI 开发→自动验证"的闭环，大幅提升迭代效率

**后续改进思路：** - 探索"分而治之"的 Prompt 策略：将复杂功能拆解为独立子任务，逐模块生成和验证，而非一次性生成全部代码 - 引入 AI 辅助的代码审查环节：让 AI 在生成代码后自我审查安全性、可访问性和浏览器兼容性 - 建立个人 Prompt 模板库：将有效的提示词模式沉淀为可复用的模板

#### 4. AI 代码生成存在的局限性，人在开发中不可替代的价值

尽管本次开发体验流畅，但 AI 有明显的局限性：

**AI 做不好的事：** - **品味判断**：AI 能生成"好看的"代码，但无法真正理解"美"——它只是在模仿训练数据中的模式。本次渐变配色方案是人工选择的（8 组预设和 5 种算法），AI 只是执行者 - **用户共情**：AI 不知道用户真正需要什么。是我判断出"渐变实验室"比"声音可视化"更适合作为课程作业（后者演示效果好但调试风险高），这种取舍需要人的经验判断 - **创造性突破**：AI 擅长组合已有模式，难以产生真正的原创想法。本次的四个选题方案本质上是已有产品形态的变体，而非全新创造

**人不可替代的价值：** 1. **需求定义者** — 只有人能真正理解"用户想要什么" 2. **质量守门员** — AI 的代码可能包含安全隐患、性能问题或过时 API，需要人来把关 3. **审美决策者** — 配色、布局、交互节奏等设计决策需要人的审美判断 4. **学习主体** — Vibe Coding 的目的是让你更快地学习和创造，而非替代你的思考和成长

Vibe Coding 不是"AI 替你写代码"，而是"你和 AI 一起写代码"——你是驾驶员，AI 是引擎。

---

以上内容基于与 AI (Hermes Agent / DeepSeek v4 Pro) 的真实对话记录撰写。